

VIETNAM NATIONAL UNIVERSITY-HO CHI MINH CITY

HO CHI MINH UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

KNOWLEDGE ENGINEERING DEPARTMENT

DVC for Data Version Control

Course: Applications of Natural Language Processing in Industry

Student:

23127165 - Nguyen Hai Dang

Lecturer:

Mr. Nguyen Hong Buu Long

Mr. Luong An Vinh

Ngày 25 tháng 3 năm 2026



Mục lục

1	Introduction	2
2	Local Setup & Core Architecture (macOS)	2
2.1	Project Initialization and Remote Configuration	2
2.2	Tracking and Pushing Version 1 (v1)	3
2.3	Updating the Dataset to Version 2 (v2)	3
3	Retrieving Dataset on Another Machine (Google Colab)	4
3.1	Step 1: Downloading Source Code	4
3.2	Step 2: Granting Access and Pulling Data	4
4	List and Switch Between Dataset Versions	5
4.1	Step 3: Listing the Versions	5
4.2	Step 4: Switching Back to Version 1	5

1 Introduction

This report documents the practical implementation of Data Version Control (DVC). Following the provided instructions, the objective was to initialize a local project tracking a sentiment dataset, connect it to remote storage (GitHub for code, DagsHub for data), and demonstrate how to manage dataset versions across different environments. As requested, this report illustrates the workflow of retrieving the dataset on another machine and switching between dataset versions, supported by screen captures.

Link Project:

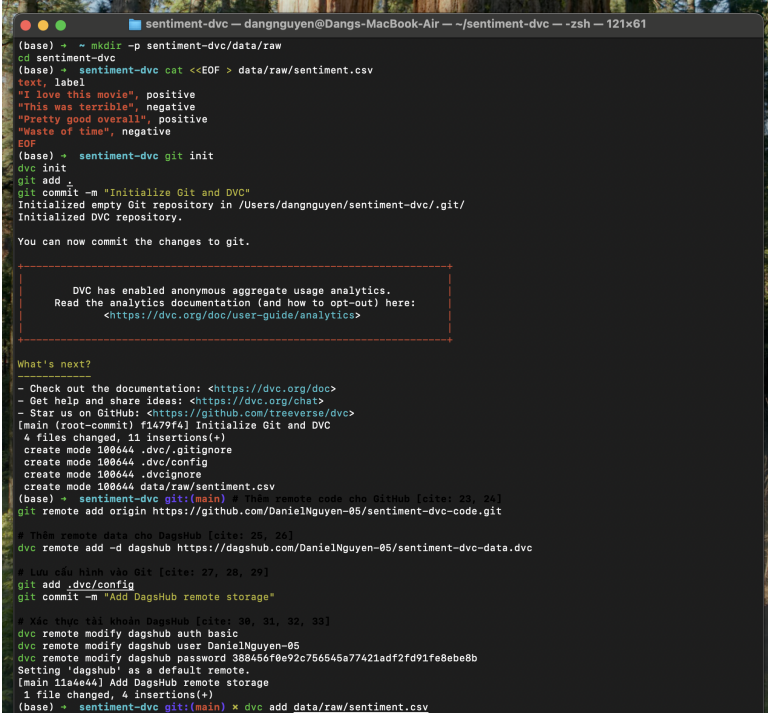
- Google Colab: [NLP_DVC](#)
- GitHub Repository: [sentiment-dvc-code](#)
- DagsHub Repository: [sentiment-dvc-data](#)

2 Local Setup & Core Architecture (macOS)

All foundational steps were executed on a local macOS Terminal to establish the version control environment.

2.1 Project Initialization and Remote Configuration

First, the required folder structure `sentiment-dvc/data/raw/` was created, and both Git and DVC were initialized. To establish remote storage, the code repository was linked to GitHub, and the data repository was connected to DagsHub. The authentication for DagsHub was also successfully configured.



```
sentiment-dvc — dangnguyen@Dangs-MacBook-Air — ~/sentiment-dvc — zsh — 121x61
(base) ~ % mkdir -p sentiment-dvc/data/raw
cd sentiment-dvc
(base) ~ % sentiment-dvc cat <<EOF > data/raw/sentiment.csv
text, label
"I love this movie", positive
"This was terrible", negative
"Pretty good overall", positive
"Waste of time", negative
EOF
(base) ~ % sentiment-dvc git init
dvc init
git add .
git commit -m "Initialize Git and DVC"
Initialized empty Git repository in /Users/dangnguyen/sentiment-dvc/.git/
Initialized DVC repository.

You can now commit the changes to git.

DVC has enabled anonymous aggregate usage analytics.
Read the analytics documentation (and how to opt-out) here:
<https://dvc.org/doc/user-guide/analytics>

What's next?
- Check out the documentation: <https://dvc.org/doc>
- Get help and share ideas: <https://dvc.org/chat>
- Star us on GitHub: <https://github.com/treasure/dvc>
[main (root-commit) 61279f6] Initialize Git and DVC
4 files changed, 11 insertions(+)
create mode 100644 .dvc/.gitignore
create mode 100644 .dvc/config
create mode 100644 .dvcignore
create mode 100644 data/raw/sentiment.csv
(base) ~ % sentiment-dvc git:(main) # Then remote code cho GitHub [cite: 23, 24]
git remote add origin https://github.com/DanielNguyen-05/sentiment-dvc-code.git

# Then remote data cho DagsHub [cite: 25, 26]
dvc remote add -d dagshub https://dagshub.com/DanielNguyen-05/sentiment-dvc-data.dvc

# Then add DagsHub to git [cite: 27, 28, 29]
git add .dvc/config
git commit -m "Add DagsHub remote storage"

# Then modify DagsHub auth info [cite: 30, 31, 32, 33]
dvc remote modify dagshub auth basic
dvc remote modify dagshub user DanielNguyen-05
dvc remote modify dagshub password 388456f0e92c756545a77421adf2fd91fe8ebe8b
Setting 'dagshub' as a default remote.
[main 124e44d] Add DagsHub remote storage
1 file changed, 4 insertions(+)
(base) ~ % sentiment-dvc git:(main) * dvc add data/raw/sentiment.csv
```

2.2 Tracking and Pushing Version 1 (v1)

After creating the initial dataset (`sentiment.csv`), it was added to DVC. I ensured that Git only tracked the lightweight `.dvc` file and configuration files, not the raw data itself. The codebase was then pushed to GitHub, and the physical dataset was successfully pushed to DagsHub, marking the establishment of Version 1.

```

sentiment-dvc — dangnguyen@Dangs-MacBook-Air — ~/sentiment-dvc — zsh — 121x61
(base) + sentiment-dvc git:(main) * git rm -r --cached 'data/raw/sentiment.csv'
git commit -m "stop tracking data/raw/sentiment.csv"
rm 'data/raw/sentiment.csv'
[main c45ad1c] stop tracking data/raw/sentiment.csv
1 file changed, 5 deletions(-)
delete mode 100644 data/raw/sentiment.csv
(base) + sentiment-dvc git:(main) * dvc add data/raw/sentiment.csv
100% Adding... |1/1 [00:00, 7.77file/s]

To track the changes with git, run:

    git add data/raw/sentiment.csv.dvc data/raw/.gitignore

To enable auto staging, run:

    dvc config core.autostage true
(base) + sentiment-dvc git:(main) * git add data/raw/sentiment.csv.dvc data/raw/.gitignore
git commit -m "Track sentiment dataset v1"
[main 2794259] Track sentiment dataset v1
2 files changed, 6 insertions(+)
create mode 100644 data/raw/.gitignore
create mode 100644 data/raw/sentiment.csv.dvc
(base) + sentiment-dvc git:(main) * git push -u origin main
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 8 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (21/21), 1.79 KiB | 1.79 MiB/s, done.
Total 21 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/DanielNguyen-05/sentiment-dvc-code.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
(base) + sentiment-dvc git:(main) * dvc push
Collecting
Pushing
1 file pushed
(base) + sentiment-dvc git:(main) * cat data/raw/sentiment.csv.dvc

```

2.3 Updating the Dataset to Version 2 (v2)

To simulate a data cleaning or updating process, a new row of data ("Not bad at all", positive) was appended to the `sentiment.csv` file. This updated dataset was tracked by DVC, generating a modified `.dvc` file. The new state was committed to Git as "Dataset v2: cleaned labels". Finally, both the updated code and the new data version were pushed to their respective remote repositories.

```

sentiment-dvc — dangnguyen@Dangs-MacBook-Air — ~/sentiment-dvc — zsh — 115x30
(base) + sentiment-dvc git:(main) * echo "Not bad at all", positive' >> data/raw/sentiment.csv
dvc add data/raw/sentiment.csv
100% Adding... |1/1 [00:00, 63.96file/s]

To track the changes with git, run:

    git add data/raw/sentiment.csv.dvc

To enable auto staging, run:

    dvc config core.autostage true
(base) + sentiment-dvc git:(main) * git add data/raw/sentiment.csv.dvc
git commit -m "Dataset v2: cleaned labels"
[main eb4852a] Dataset v2: cleaned labels
1 file changed, 2 insertions(+), 2 deletions(-)
(base) + sentiment-dvc git:(main) * git push
dvc push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 486 bytes | 486.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/DanielNguyen-05/sentiment-dvc-code.git
 2794259..eb4852a  main -> main
Collecting
Pushing
1 file pushed
(base) + sentiment-dvc git:(main) *

```

3 Retrieving Dataset on Another Machine (Google Colab)

To fulfill the requirement of retrieving the dataset on another machine, Google Colab was used to simulate a fresh remote environment.

3.1 Step 1: Downloading Source Code

The code repository was cloned directly from GitHub. This step imports the Git history and the .dvc tracking files, but not the actual data.

```
✓ Task 1: Retrieve dataset on another machine (Google Colab)

✓ Step 1: Download the source code from GitHub to Colab.

[2]
✓ 0s !git clone https://github.com/DanielNguyen-05/sentiment-dvc-code.git
    %cd sentiment-dvc-code

✓ Cloning into 'sentiment-dvc-code'...
    remote: Enumerating objects: 26, done.
    remote: Counting objects: 100% (26/26), done.
    remote: Compressing objects: 100% (16/16), done.
    remote: Total 26 (delta 3), reused 26 (delta 3), pack-reused 0 (from 0)
    Receiving objects: 100% (26/26), done.
    Resolving deltas: 100% (3/3), done.
    /content/sentiment-dvc-code
```

3.2 Step 2: Granting Access and Pulling Data

Next, I configured the authentication for DagsHub using my credentials. Finally, I executed the `dvc pull` command. DVC read the local .dvc file and downloaded the actual physical data (version 2) from DagsHub. Running the `cat` command confirmed that the latest dataset containing 5 rows was successfully retrieved.

```
✓ Step 2: Grant Colab access to DagsHub and download the data.

[3]
✓ 7s # Authentication Configuration
    !dvc remote modify dagshub auth basic
    !dvc remote modify dagshub user DanielNguyen-05
    !dvc remote modify dagshub password 388456f0e92c756545a77421adf2fd91fe8ebe8b

    # Pull actual data (latest version - v2) from DagsHub
    !dvc pull

    # Print file contents to prove data v2 has been retrieved
    !cat data/raw/sentiment.csv

✓ Collecting      |1.00 [00:00, 58.5entry/s]
    Fetching
    !
    0% |          |0/? [00:00<?,    ?files/s]
    Fetching
    Building workspace index      |3.00 [00:00, 754entry/s]
    Comparing indexes            |4.00 [00:00, 986entry/s]
    Applying changes             |0.00 [00:00,    ?file/s]
    Everything is up to date.
    text, label
    "I love this movie", positive
    "This was terrible", negative
    "Pretty good overall", positive
    "Waste of time", negative
```

4 List and Switch Between Dataset Versions

The final requirement is to demonstrate how to list and switch between dataset versions.

4.1 Step 3: Listing the Versions

Using the `git log --oneline` command, I reviewed the commit history of the repository. This provided the specific commit hashes for the different dataset versions (e.g., the commit for "Dataset v2: cleaned labels" and "Track sentiment dataset v1").

```
Task 2: List and Switch between dataset versions

Step 3: List the versions

[4]
✓ 0s !git log --oneline

eb4852a (HEAD -> main, origin/main, origin/HEAD) Dataset v2: cleaned labels
2794259 Track sentiment dataset v1
c45ad1c stop tracking data/raw/sentiment.csv
11a4e44 Add DagsHub remote storage
f1479f4 Initialize Git and DVC
```

4.2 Step 4: Switching Back to Version 1

To revert the dataset to its initial state, a two-step process was required:

1. **Git Checkout:** I ran `git checkout 2794259 data/raw/sentiment.csv.dvc` to revert the specific tracking file back to version 1.
2. **DVC Checkout:** I ran `dvc checkout` to command DVC to synchronize the actual physical data file with the updated tracking file.

Running the `cat` command afterward proved that the operation was successful, as the `sentiment.csv` file was restored to its original 4-row state.

```
Step 4: Switch back to version v1 (Switch versions)

[5]
✓ 1s # Restore the tracking file .dvc to version v1
!git checkout 2794259 data/raw/sentiment.csv.dvc

# Request DVC to synchronize the actual data according to the modified .dvc file
!dvc checkout

# Print the contents for verification
!cat data/raw/sentiment.csv

Updated 0 paths from d085477
Building workspace index |3.00 [00:00, 19.3entry/s]
Comparing indexes |4.00 [00:00, 612entry/s]
Applying changes |0.00 [00:00, ?file/s]
text, label
"I love this movie", positive
"This was terrible", negative
"Pretty good overall", positive
"Waste of time", negative
```